5. Folosind cunoștiințele de curs, răspundeți la următoarele întrebări scurte:

(a) care este cel mai mare număr zecimal (natural) care se poate reprezenta pe N biți?

(b) care este cel mai mare număr întreg care se poate reprezenta (complement față de doi) pe N biți? dar cel mai mic?

(c) fie x un număr natural, de câți biți avem nevoie să reprezentăm x în sistemul binar?

(d) dacă un număr x este reprezentat cu k cifre în sistemul hexazecimal, de câți biți avem nevoie pentru a reprezenta x în sistemul binar?

(e) dacă un număr x este reprezentat cu k biți în sistemul binar, de câte cifre avem nevoie pentru a reprezenta x în sistemul hexazecimal?

(f) dacă un număr x este reprezentat cu k cifre în sistemul zecimal, de câți biți avem nevoie pentru a reprezenta x în sistemul binar?

a) $2^N - 1$

b) $2^{N-1} - 1$ și -2^{N-1}

c) aproximativ $\log_2 x$, exact sunt ceil($\log_2 (x+1)$)

d) 4k

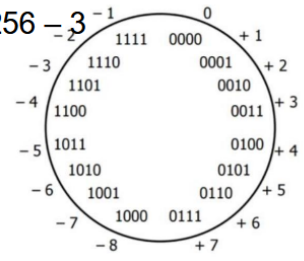
e) ceil (k / 4)

f) ceil (k $\log_2 10$)

bit b_i:	1	1	1	1	0	0	0	1
2^i:	-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

- $x = -b_{N-1}2^{N-1} + \sum_{i=0}^{N-2} b_i 2^i$

- **ca să reprezentăm un număr negativ, luăm valoarea pozitivă a numărului, îi inversăm biții și adunăm unu**
- **de ce funcționează această procedură?**
  - pornim de la faptul că folosim aritmetică modulo
  - fixăm și suntem pe 8 biți
  - deci, să scădem 3 e echivalent cu a aduna 256 – 3
  - -3 ≡ 256 – 3 = 1 0000 0000 – 0000 0011
    = 1 + 1111 1111 – 0000 0011
    = 1 + (3 cu biții inversați)



```
int binarySearch1(int arr[], int start, int end, int x)
{
        if (end >= start)
        {
                int mid = start + (end - start) / 2;

                if (arr[mid] == x)
                        return mid;

                if (arr[mid] > x)
                        return binarySearch1(arr, start, mid - 1, x);

                return binarySearch1(arr, mid + 1, end, x);
        }

        return -1;
}

int binarySearch2(int arr[], int start, int end, int x)
{
        if (end >= start)
        {
                int mid = (start + end) / 2;

                if (arr[mid] == x)
                        return mid;

                if (arr[mid] > x)
                        return binarySearch2(arr, start, mid - 1, x);

                return binarySearch2(arr, mid + 1, end, x);
        }

        return -1;
}
```

Binary search 1 nu produce overflow

Seminar 01

$$H(X) = E(I(X)) = \sum_{i=1}^{N} p_i \log_2 \frac{1}{p_i} = \sum_{i=1}^{N} -p_i \log_2 p_i$$

- **în urnă: 5 bile roșii, 3 bile albastre**
- **observăm la extragere o bilă albastră, câtă informație am primit?**

$$I(\text{bila albastra}) = \log_2\left(\frac{1}{\frac{3}{8}}\right) = \log_2\left(\frac{8}{3}\right) = 1.42 \text{ biti}$$

- **cât era entropia urnei inainte de extragere?**

$$H(\text{urna}) = \frac{5}{8}\log_2\left(\frac{8}{5}\right) + \frac{3}{8}\log_2\left(\frac{8}{3}\right) = 0.95 \text{ biti}$$

- **cât este entropia urnei după extragere?**

$$H(\text{urna dupa extragere}) = \frac{5}{7}\log_2\left(\frac{7}{5}\right) + \frac{2}{7}\log_2\left(\frac{7}{2}\right) = 0.86 \text{ biti}$$

Se dau 12 bile. 11 dintre ele au aceeași greutate, iar una este mai ușoară sau mai grea decât toate celelalte. Aveți la dispoziție o balanță. Folosind un număr minim de cântăriri, găsiți bila diferită.

- **12 bile, una dintre ele are o greutate diferită față de celelalte**
- **găsiți bila diferită cu un număr minim de cântăriri**

- **întrebări inițiale:**
  - câte cântăriri avem nevoie cu soluția “simplă”?
    - cântărim două câte două bilele (6 experimente): un experiment va reduce opțiunile la două bile, apoi comparăm fiecare bilă cu una din cele care es cu siguranță standard (2 experimente)
    - deci cu 8 cântăriri putem sigur găsi bila
  - câtă informație trebuie să descoperim?
    - care e bila diferită? $\log_2(12) = 3.58$
    - dacă este mai ușoară sau mai grea: 1 bit, dacă nu știm care situație este

considerăm cazul mai simplu, în care știm că bila diferită e mai grea/ușoară

- **să analizăm sistematic**
  - avem 3 valori ce trebuie specificate când facem o cântarire
    - câte bile putem în stanga balanței
    - câte bile putem în dreapta balanței
    - câte bile rămân pe masă
  - toate posibilitățile care ne interesează:
    - 6 cu 6, 0 pe masă
    - 5 cu 5, 2 pe masă
    - 4 cu 4, 4 pe masă
    - 3 cu 3, 6 pe masă
    - 2 cu 2, 8 pe masă
    - 1 cu 1, 10 pe masă

Avem un alfabet cu două simboluri: simbolul "A" este codat pe un singur bit și are probabilitatea de apariție p și simbolul "B" este codat pe doi biți și are probabilitatea de apariție $1-p$ (bineînțeles).

Cerințe:

(a) calculați lungimea medie a mesajelor cu alfabetul de mai sus;

(b) definim rata entropiei ca fiind entropia supra lungimea medie a mesajelor. Găsiți p pentru care rata entropiei este minimizată/maximizată.

- **lungimea medie a mesajului**
  - 1xp + 2x(1-p) = 2 – p

- **entropia în cazul nostru**

$$H = p\log_2\frac{1}{p} + (1-p)\log_2\frac{1}{1-p}$$

- **rata entropiei**

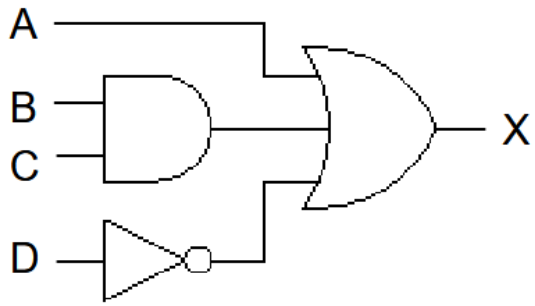
$$R = \frac{p\log_2\frac{1}{p} + (1-p)\log_2\frac{1}{1-p}}{2-p}$$

- **cum maximizăm?**
  - derivăm $R' = \frac{2\log_2\frac{1}{p} - \log_2\frac{1}{1-p}}{(2-p)^2}$ și egalăm cu zero $\longrightarrow$ $\log_2\frac{1-p}{p^2} = 0$
  - p este soluția ecuației $p^2 + p - 1 = 0 \longrightarrow p = \frac{\sqrt{5}-1}{2}$

care este legătura cu raportul de aur?

DESENAȚI CIRCUITUL, EX 2

X = A + BC + $\bar{D}$



Legile de Morgan sunt: $\overline{A+B} = \bar{A}\bar{B}$ și $\overline{AB} = \bar{A} + \bar{B}$. Cerințe:

- **numarul nostru x este**



- **deplasare normală cu 2 la dreapta >>**



echivalent cu o împărțire la 2^2

- **deplasare aritmetică cu 2 la dreapta $>>_a$**



echivalent cu o împărțire la 2^2

- **deplasare circulară cu 2 la dreapta $>>_c$**



Toate deplasările pot fi definite pentru orice bază numerică, nu doar 2. De exemplu: în baza 10 avem $754 <<_c 2 = 475$ pentru deplasare circular la stânga și $754 << 2 = 400$ pentru deplasare normală la stânga (pe 3 cifre)

- 13. Răspundeți la următoarele întrebări despre formatul IEEE 754 FP:

  (a) câte numere distincte se pot reprezenta cu acest format?
  (b) câte numere distincte în intervalul 2^{-5} și 2^{-4} se pot reprezenta cu acest format?
  (c) câte numere distincte în intervalul 2^{13} și 2^{14} se pot reprezenta cu acest format?
  (d) câte numere distincte în intervalul 2^{47} și 2^{49} se pot reprezenta cu acest format?

  (a) numere distincte. (510*2^23+1)

- **(b)** 2^23
- **(c)** 2^23
- **(d)** 2^24

1. Presupunem ca avem un sistem de calcul pe 32 de biți, răspundeți la următoarele întrebări scurte:

   (a) care este adresa de memorie cea mai mare care poate fi accesată? (cu 8 biți / locație memorie)

   (b) avem instrucțiunea *jne etichetă*, unde *jne* are opcode-ul 0110. Care este saltul maxim care se poate realiza cu această instrucțiune?

   (c) avem o instrucțiune *add R1, R2*, unde *add* are opcode-ul 0011 iar *R1* și *R2* sunt regiștri iar calculul realizat este $R2 \leftarrow R1 + R2$. Câți regiștri diferiți putem avea?

   (d) similar cu instrucțiunea anterioară, dar acum avem *add R1, R2, R3*, unde add are opcode-ul 0100 iar calculul realizat este $R3 \leftarrow R1 + R2$. Câți regiștri diferiți putem avea?

a) adresa de memorie cea mai mare accesibilă este 2^{32} = 4GB (4.294.967.296 bytes)

b) instrucțiunea este *jne etichetă*, unde *jne* are opcode 0110

- adresa de memorie cea mai mare accesibilă este 2^{28} = 0.25 GB

c) avem instrucțiunea add R1, R2

- opcode este 0011
- operația suportă 2^{14} = 16384 regiștri diferiți

d) avem instrucțiunea add R1, R2, R3

- opcode este 0100
- vom avea 9.33 biți pentru fiecare reprezentare a unui registru: două poziții suportă 2^9 = 512 iar o poziție 2^{10} = 1024

* Sistemele moderne sunt pe 64 biți, 2^{64} este 18,446,744,073,709,551,616 (adică 17,179,869,184 GB)

a) %eax ← %ebx + %ecx
%eax ← %ebx + %edx WAW

b) %ebx ← %ecx + %eax
%eax ← %edx + %eax WAR

c) %eax ← %ebx + %ecx
%edx ← %eax + %edx RAW

d) %eax ← 6
%eax ← 3 WAW
%ebx ← %eax + 7 RAW , rezultatul poate fi 10 sau 13

BRANCH PREDICTION, EX. 6

- **ce face algoritmul?**
  - merge (interclasare)
- **câte instrucțiuni de salt avem?**
  - 4
- **predicția pentru fiecare?**
  - Salt 1: sare mereu
  - Salt 2: în general, nu știm
  - Salt 3: sare mereu
  - Salt 4: sare mereu
- **cum eliminăm Saltul 2?**
  - int cmp = (*A <= *B)
  - int min = *B ^ ((*B ^ *A) & (-cmp))
    - *C++ = min
    - A += cmp, na -= cmp
    - B += !cmp, nb -= !cmp

```
while (na > 0 && nb > 0)
{
    if (*A <= *B) {
        *C++ = *A++; --na;
    } else {
        *C++ = *B++; --nb;
    }
}

while (na > 0) {
    *C++ = *A++; --na;
}

while (nb > 0) {
    *C++ = *B++; --nb;
}
```

- **se poate mai eficient?**
  - loop unrolling

```
int sum = 0;
int i = 0;
for (i = 0; i < 10; i+=2) {
    sum += i;
    sum += i+1;
}
```

  - de ce am vrea să facem așa ceva?
    - mai puține salturi

2. Presupunem că avem un sistem de calcul despre care vrem să știm anumite lucruri (legate de performanța sa). În scenarii diferite, ne interesează criterii de performanță diferite, fiecare având o metodă proprie de estimare. Rulăm aceleași programe de mai multe ori și trebuie să decidem care este funcția de agregare (media artimetică și geometrică, mediana, maximum și minimum). Ce criteriu de performanță am măsura și cum l-am estima în următoarele cazuri:

   (a) ne interesează să putem răspunde la cât mai multe cereri de date venite de pe Internet;

   (b) vrem să ne asigurăm că cererile de date sunt finalizate în 10ms;

   (c) ne interesează ca cererile să ocupe maxim 100MB în memorie;

   (d) vrem să rulăm sistemul de calcul cu un cost cât mai mic;

   (e) vrem să știm că majoritatea cererilor sunt servite în maximum 50ms;

   (f) vrem să estimăm timpul total de răspuns al sistemului;

   (g) este vreodată folositor să estimăm performanța folosind minimul/maximul?

   (h) de ce este foarte important să măsurăm cât mai bine/exact performanța unui sistem de calcul?

   În general, un sistem de calcul trebuie să îndeplinească un anumit standard de funcționare (Quality of Service QoS sau Service Level Agreement SLA) care implică mai multe (de obicei cel puțin 2–3) dintre criteriile de performanță enumerate de mai sus.

a) utilizare CPU (eventual sisteme multi-core), media artimetică

b) wall-clock time, media aritmetică

c) memoria RAM, maximum

d) performanță per Watt, media artimetică sau maximum

e) wall-clock time, 50/90/99th percentile mediana

f) wall-clock time speedup, media aritmetică

g) minimum = când zgomotul/erorile din sistem sunt minime (*best-case behavior*), maximum = când zgomotul/erorile din sistem sunt maxime (*worst-case behavior*)

h) dacă măsurăm cât mai bine și exact fiecare componentă, putem optimiza cât mai bine (semnificativ)

4. Se dau două numere complexe $x = a + bi$ și $y = c + di$. Răspundeți la următoarele întrebări:

   (a) scrieți explicit formula pentru $z = x \times y$;

   (b) câte adunări și înmulțiri se realizează pentru calculul lui z?

   (c) puteți să calculați z cu mai puține înmulțiri?

   (d) de câte ori ar trebui să fie mai lentă o înmulțire față de a adunare pentru ca rezultatul de la punctul precedent să fie eficient?

   (e) ideea de a înlocui o înmulțire cu mai multe adunări (în general, o operație dificilă cu o serie de operații simple) apare de mai multe ori în algoritmică (vedeți algoritmul lui Strassen).

e) algoritmul lui Strassen

- vrem să calculăm C = AB (unde A și B sunt matrice)

```
for (i = 0; i < row_length_A; i++)
{
    for (k = 0; k < column_length_B; k++)
    {
        sum = 0;
        for (j = 0; j < column_length_A; j++)
        {
            sum += A[i][j] * B[j][k];
        }
        C[i][k] = sum;
    }
}
```

- pe blocuri:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} \end{bmatrix}, \mathbf{B} = \begin{bmatrix} \mathbf{B}_{1,1} & \mathbf{B}_{1,2} \\ \mathbf{B}_{2,1} & \mathbf{B}_{2,2} \end{bmatrix}, \mathbf{C} = \begin{bmatrix} \mathbf{C}_{1,1} & \mathbf{C}_{1,2} \\ \mathbf{C}_{2,1} & \mathbf{C}_{2,2} \end{bmatrix}$$

$$\begin{aligned}
\mathbf{C}_{1,1} &= \mathbf{A}_{1,1}\mathbf{B}_{1,1} + \mathbf{A}_{1,2}\mathbf{B}_{2,1} \\
\mathbf{C}_{1,2} &= \mathbf{A}_{1,1}\mathbf{B}_{1,2} + \mathbf{A}_{1,2}\mathbf{B}_{2,2} \\
\mathbf{C}_{2,1} &= \mathbf{A}_{2,1}\mathbf{B}_{1,1} + \mathbf{A}_{2,2}\mathbf{B}_{2,1} \\
\mathbf{C}_{2,2} &= \mathbf{A}_{2,1}\mathbf{B}_{1,2} + \mathbf{A}_{2,2}\mathbf{B}_{2,2}
\end{aligned}$$

$O(n^3)$

$$\begin{aligned}
\mathbf{M}_1 &:= (\mathbf{A}_{1,1} + \mathbf{A}_{2,2})(\mathbf{B}_{1,1} + \mathbf{B}_{2,2}) \\
\mathbf{M}_2 &:= (\mathbf{A}_{2,1} + \mathbf{A}_{2,2})\mathbf{B}_{1,1} \\
\mathbf{M}_3 &:= \mathbf{A}_{1,1}(\mathbf{B}_{1,2} - \mathbf{B}_{2,2}) \\
\mathbf{M}_4 &:= \mathbf{A}_{2,2}(\mathbf{B}_{2,1} - \mathbf{B}_{1,1}) \\
\mathbf{M}_5 &:= (\mathbf{A}_{1,1} + \mathbf{A}_{1,2})\mathbf{B}_{2,2} \\
\mathbf{M}_6 &:= (\mathbf{A}_{2,1} - \mathbf{A}_{1,1})(\mathbf{B}_{1,1} + \mathbf{B}_{1,2}) \\
\mathbf{M}_7 &:= (\mathbf{A}_{1,2} - \mathbf{A}_{2,2})(\mathbf{B}_{2,1} + \mathbf{B}_{2,2})
\end{aligned}$$

$$\begin{aligned}
\mathbf{C}_{1,1} &= \mathbf{M}_1 + \mathbf{M}_4 - \mathbf{M}_5 + \mathbf{M}_7 \\
\mathbf{C}_{1,2} &= \mathbf{M}_3 + \mathbf{M}_5 \\
\mathbf{C}_{2,1} &= \mathbf{M}_2 + \mathbf{M}_4 \\
\mathbf{C}_{2,2} &= \mathbf{M}_1 - \mathbf{M}_2 + \mathbf{M}_3 + \mathbf{M}_6
\end{aligned}$$

$O(n^{2.8})$

a) $z = (a+bi)x(c+di) = ac - bd + i(ad + bc)$

b) 2 adunări, 4 înmulțiri

c) calculăm $S1 = ac$, $S2 = bd$ și $S3 = (a+b)x(c+d)$

$z = S1-S2 + i(S3-S1-S2)$

5 adunări, 3 înmulțiri

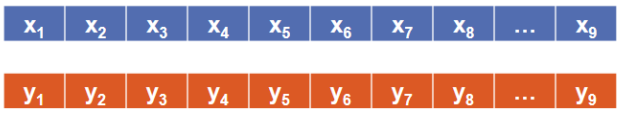
d) C1 – costul unei adunări

C2 – costul unei înmulțiri

$2C1 + 4C2 > 5C1 + 3C2$

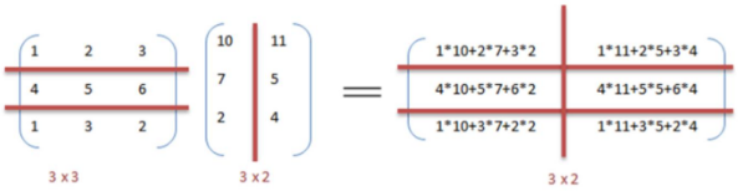
$C2/C1 > 3$

- **produs scalar**



- **cum calculăm eficient?**
  - $c \mathrel{+}= x_i\, y_i$
  - instrucțiune Fast Multiply-Add (fma)
  - aceeași operație pe date diferite (SIMD)
  - foarte ușor de paralelizat
    - atenție, rezultatul va fi diferit (adunarea nu mai e asociativă)
    - cu *p* procesoare ne așteptăm să fim de *p* ori mai rapizi
  - exploatează cache la maxim: datele sunt continue în memorie

- **produs matrice-matrice**



- **cum calculăm eficient?**
  - c += x_i y_i
  - n^2 produse scalare (se pot realiza în paralel)
  - cum exploatăm eficient cache-ul?
    - calcul pe blocuri, nu pe linii sau coloane

```
for (ii = 0; ii < SIZE; ii += BLOCK_SIZE)
  for (kk = 0; kk < SIZE; kk += BLOCK_SIZE)
    for (jj = 0; jj < SIZE; jj += BLOCK_SIZE)
      maxi = min(ii + BLOCK_SIZE, SIZE);
      for (i = ii; i < maxi; i++)
        maxk = min(kk + BLOCK_SIZE, SIZE);
        for (k = kk; k < maxk; k++)
          maxj = min(jj + BLOCK_SIZE, SIZE);
          for (j = jj; j < maxj; j++)
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
```

```
# varianta A
for (int i = 0; i < n; ++i)
  for (int j = 0; j < n; ++j)
     for (int k = 0; k < n; ++k)
        C[i][j] += A[i][k] * B[k][j];

# varianta B
for (int j = 0; j < n; ++j)
  for (int i = 0; i < n; ++i)
     for (int k = 0; k < n; ++k)
        C[i][j] += A[i][k] * B[k][j];

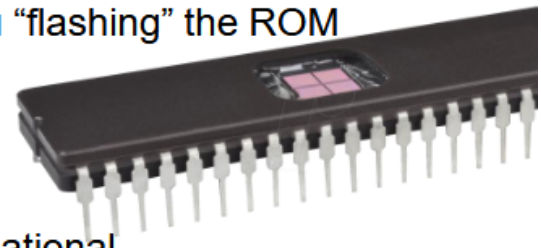
# varianta C
for (int k = 0; k < n; ++k)
  for (int i = 0; i < n; ++i)
     for (int j = 0; j < n; ++j)
        C[i][j] += A[i][k] * B[k][j];
```

C>A>B

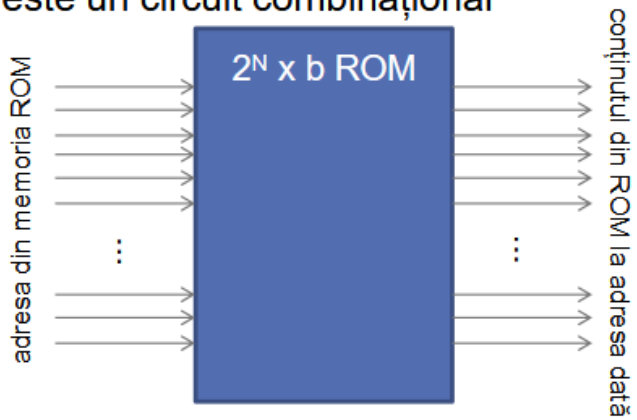
B = cache miss(pe coloane)

- **pornirea sistemului**
  - în general, un buton de “power on” / “power off” cum funcționează un astfel de buton? atât la pornire cât și la oprire?
  - realizează alimentarea cu electricitate a componentelor
  - **CPU** este activat
  - CPU caută/pornește **BIOS** (Basic Input Output System)
    - testează componentele hardware (RAM, I/O, HD, etc.)
    - BIOS este scris în **ROM** (Read Only Memory) pe placa de bază
    - este scris într-un tip de memorie nevolatilă
    - pentru execuție, BIOS-ul este încărcat în RAM
    - BIOS știe cât e ceasul (**CMOS Real-Time Clock**) și hardware-ul
    - îl accesați automat când porniți calculatorul, fie cu F2 (în general)
  - CPU/BIOS pornesc Boot Code (caută sistemul de operare)
    - sistemul de operare este în general pe HD (poate fi și pe CD, stick)
    - sistemul de operare este încărcat în RAM pentru execuție

- **BIOS**
  - este scris în ROM (Read Only Memory)
  - câteodată în Programmable ROM / Erasable Programmable ROM / Electrically Erasable Programmable ROM
  - să scriem în ROM: “burning” sau “flashing” the ROM
  - este un tip de *firmware*

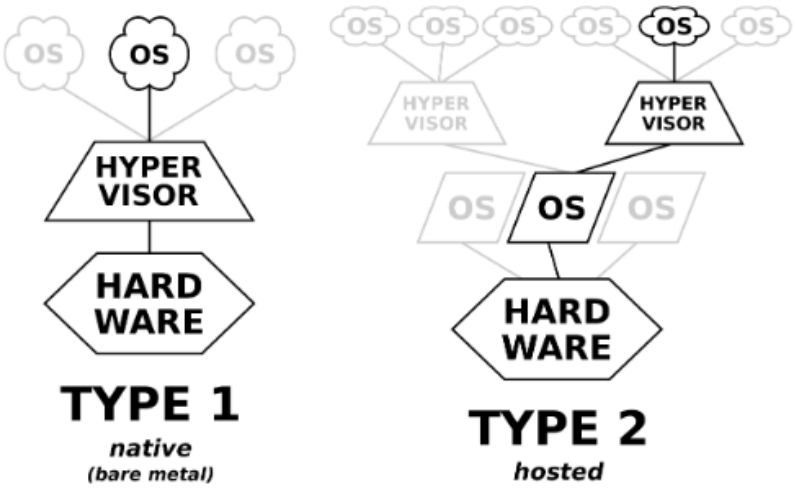


  - în general este un circuit combinațional



- **OS-ul preia controlul de la BIOS**
  - din acest moment doar OS-ul are acces direct la periferice
  - accesul este realizat prin *drivere*

  - virtualizare (hardware)
  - emulare



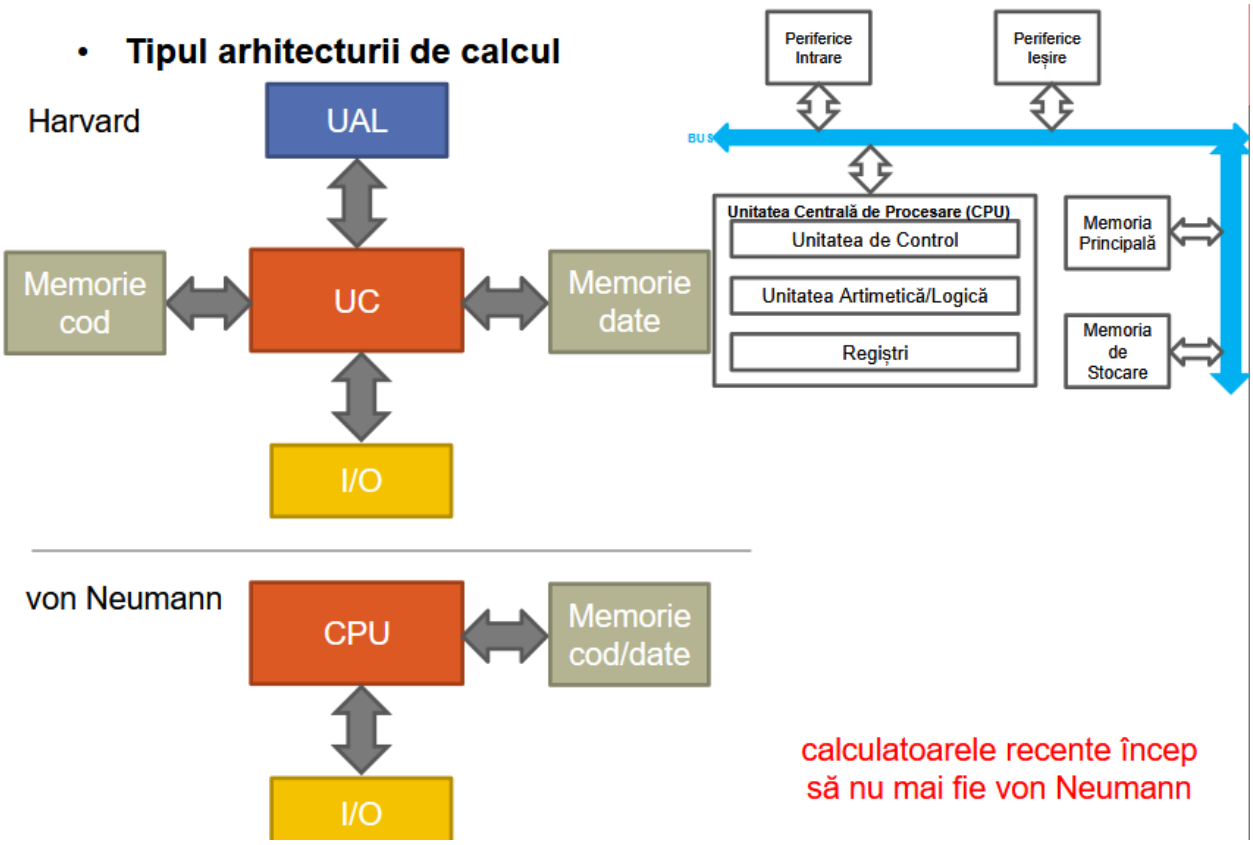
  - containere (dockere)

- **OS-ul preia controlul de la BIOS**
  - din acest moment doar OS-ul are acces direct la periferice
  - accesul este realizat prin *drivere*
  - OS-ul oferă o imagine abstractizată a memoriei pentru fiecare proces pornit

  - din momentul în care OS-ul pornește, sistemul de calcul intră în ciclul obișnuit de procesare (secvența de boot s-a terminat)

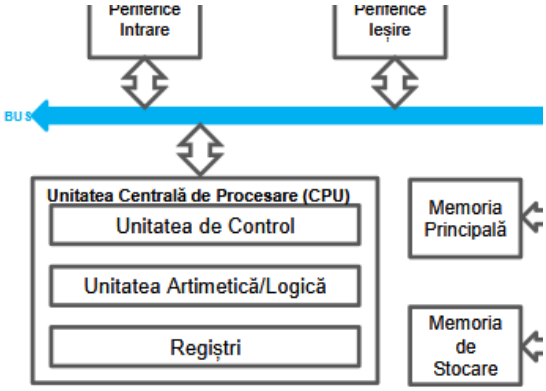
- **un sistem de calcul trebuie să fie capabil:**
  - să calculeze
    - să execute instrucțiuni
  - să comunice
    - să transfere biți între componente electronice
  - să stocheze
    - date care să fie folosite de instrucțiuni
    - instrucțiuni pentru execuție

- **Tipul arhitecturii de calcul**

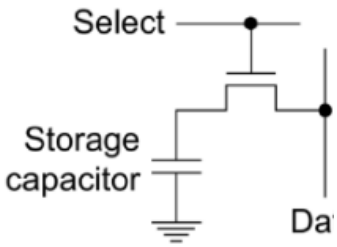


calculatoarele recente încep să nu mai fie von Neumann

- **Memoria Principală**
  - **conține cod și date**
  - **este volatilă**

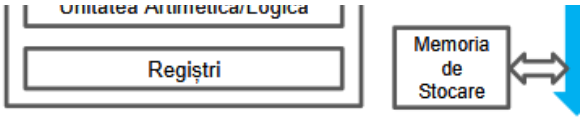


  - **Static RAM (SRAM)**
    - **bazată pe flip-flops**
    - **rapid**
    - **scump**
    - **regiștrii din CPU sunt de același tip**
  - **Dynamic RAM (DRAM)**
    - **fiecare bit este reprezentat de o combinație tranzistor + condensator**
    - **condensatoarele suferă de leakeage (scurgeri de tensiune)**
    - **DRAM trebuie actualizat o dată la fiecare câteva zeci de ms**

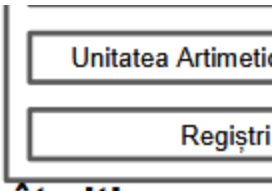


de ce este DRAM mai ieftin decât SRAM?
are DRAM niște dezavantaje în comparație cu SRAM?

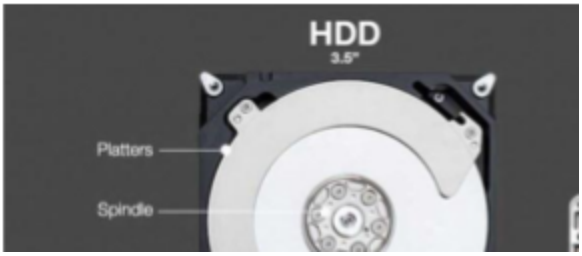
- **DDR RAM**
  - **Double Data Rate RAM**
  - **.../DDR4/DDR5/DDR6**
  - **performanța este definită de:**
    - **capacitate**
    - **dacă au un sistem intern de corectarea erorilor (ECC)**
    - **timpi de acces (în cât timp de la comanda de citire de biți din RAM avem datele disponibile?, timpul de refresh)**
    - **consumul de energie**



- **SSD (Solid State Disks)**
  - **e memorie flash, rapidă**
  - **azi, e scumpă**
  - **scrierea e mult mai lentă decât citirea**



- **HDD (Hard Disks)**
  - **mecanic**



- **BUS**
  - **conectează CPU/memorie**
  - **proprietăți**
    - **capacitatea (bandwidth)**
    - **viteza (MHz)**

- **trei mecanisme pentru execuție cu viteză sporită**
  - pipelining (conductă de date)
  - branch prediction (predicția salturilor)
  - out of order execution (execuția în ordine arbitrară)

- **CPU execută instrucțiuni:**
  - IF – Instruction Fetch
  - ID – Instruction Decode
    - COA – Calculate Operand Address
    - FO – Fetch Operand
    - FOs – Fetch Operands
  - EX – Execution
  - MEM – Memory Access
  - WB – Write Back



PIPELINING

- **este un tip de paralelism la nivel de instrucțiune (Instruction Level Paralellism - ILP)**
  - asta pentru că e paralelism diferit față de “task/process level paralellism” (thread-uri și procese)

superscalar, hardware dublat,
poate termina mai mult de o
instrucțiune într-un ciclu

- **suplimentar, situația e complicată de faptul că instrucțiuni diferite au nevoie de un număr diferit de cicli de CPU**

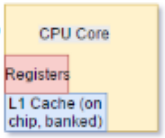
operația	instrucțiuni	# cicli
operații întregi/biți	add, sub, and, or, xor, sar, sal, lea, etc.	1
înmulțirea întregilor	mul, imul	3
împărțirea întregilor	div, idiv	depinde (20–80)
adunare floating point	addss, addsd	3
înmulțire floating point	mulss, mulsd	5
împărțire floating point	divss, divsd	depinde (20–80)
fused-multiply-add floating point	vfmass, vfmasd	5

avem regiștrii speciali

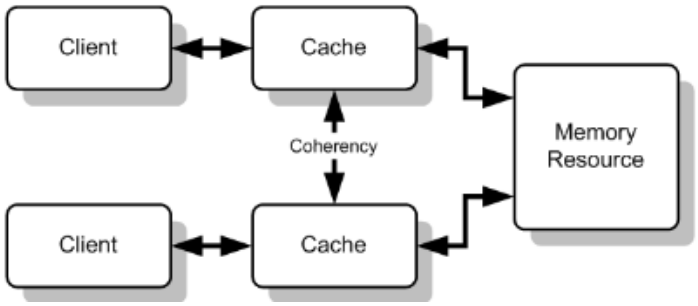
- **când complicăm hardware și software pot apărea probleme**
- **în special probleme de securitate**
  - meltdown, spectre
  - aceste două atacuri exploatează execuția speculativă și sistemul ierarhic al memoriei (cache-ul)

- **când complicăm hardware, e cu atât mai rău**
  - soluția: trebuie înlocuit hardware-ul
  - soluția: sistemul de operare trebuie să ia în considerare problema
    - *totul* va fi mai lent

- **tipuri de memorie**
  - regiștrii procesorului: acces imediat, 100-1000 bytes
  - cache L0: acces foarte rapid, 5-20 kbytes
  - cache L1 (cache instrucțiuni și date): 700GB/s, 100-500 kbytes
  - cache L2: 200GB/s, 500-1000 kbytes
  - cache L3 (de obicei partajat): 100GB/s, 1-5 MB
  - memoria principală RAM: 100-500 MB/s, 2-64 GB



  - disc HD/SSD: 10-100 MB/s, 1TB
- **corespondența dintre locația din cache și locația din RAM**
  - la citire nu sunt niciodată probleme
  - la scriere lucrurile se complică
    - rezultatul este scris în cache
    - și celelalte memorii trebuie să fie anunțate de noua valoare
      - L1/L2/L3/RAM etc.
    - situația se complică și mai mult dacă sunt cache-uri diferite pentru fiecare core pe care îl avem: cache coherence (protocol pentru consistența tuturor cache-urilor)



- **când programați, nu vedeți această ierarhizare**

- **cine e responsabil de ce anume?**
  - programatorul: transfer între HD/SSD și RAM (citire de pe disc)
  - logică hardware: din/în RAM în/din memoriile cache
  - compilatorul: generează cod care exploatează cache-ul

- **cât timp performanța este acceptabilă totul e OK, apoi Assembly**

SYMMETRIC MULTIPROCESS SYSTEMS

- **UMA (Uniform Memory Access)**
- **procese diferite pe procesoare diferite, dar e nevoie de modificarea programelor ca acestea să ruleze paralel**
- **dezavantaj: cache coherence**

ASYMMETRIC MULTIPROCESS SYSTEMS

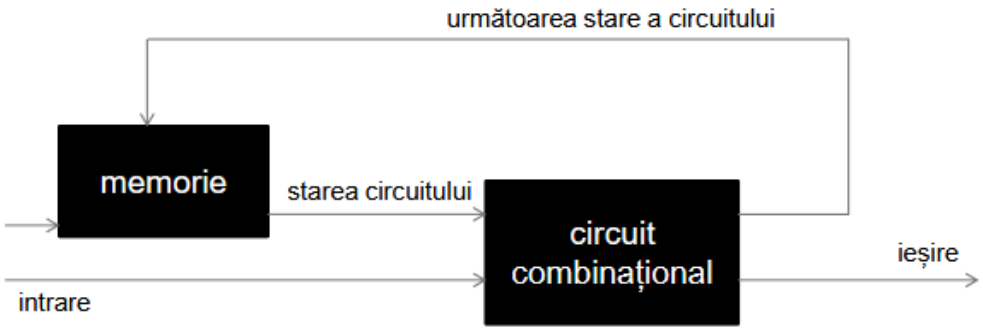
- **fiecare procesor are se ocupă de ceva diferit**
  - unul execute programe
  - altul se ocupă de I/O

NON-UNIFORM MEMORY ACCESS

- **rezolvă problema accesului la memorie de către procesoare multiple (fiecare procesor are memoria/cache-ul său)**
- **procesoarele așteaptă mai puțin să ajunge datele la ele**

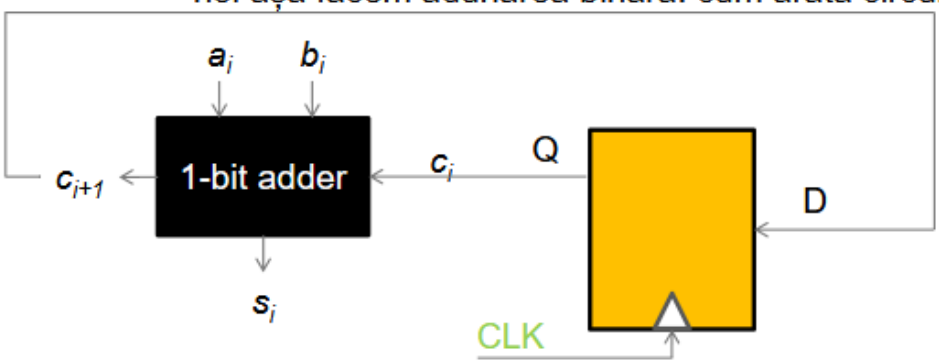
CIRCUITE SECVENȚIALE

- **avem nevoie să introducem elemente de tip “memorie”**



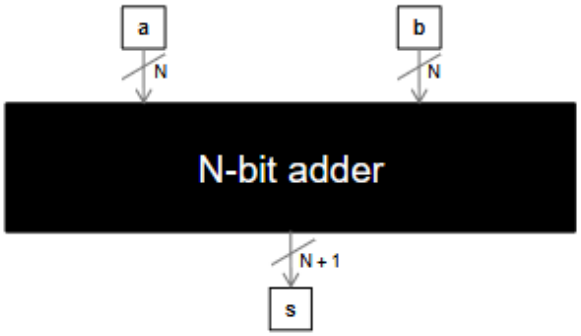
- **am vorbit până acum că biții pe care îi reprezentăm sunt voltaj**
- **dar energia electrică este dificil de stocat (în timp)**
  - fenomen de scurgere (leakage)
- **dacă vrem să memorăm ceva, trebuie să facem un refresh din când în când pentru a actualiza nivelul de energie electrică**

- **avantajele circuitelor secvențiale**
  - avem stare internă
    - exemplu: avem un registru în care memorăm count-ul curent
  - avem variabila de timp
    - intrările/ieșirile nu sunt fixe
    - număr variabil de pași în rezolvare
    - exemplu: construiți un circuit care poate să adune două numere de dimensiune (număr de biți) variabil – adică nu prestabilim pe câți biți e fiecare număr
      - noi așa facem adunarea binară. cum arată circuitul?



- **circuitele combinaționale sunt eficiente (în general) dar au o problema majoră: sunt one-shot**
  - nu putem itera
  - nu permit niciun fel de “logică internă” sau “memorie internă”
  - nu există o stare internă a circuitului
  - sunt prea simple, asociază o funcție logică a intrărilor cu o ieșire (după un anumit timp)
  - unele lucruri nu pot fi implementate folosind doar logică combinațională

- **circuit de adunare**
  - se dau *a* și *b* pe *N* biți
  - vrem să calculăm *s* = *a* + *b*
    - pe câți biți este *s*?
    - *N+1* biți



    - câte intrări avem? 2N
    - câte ieșiri? N + 1
    - cât de mare va fi circuitul combinațional? $(N+1)2^{2N-1}$

un computer care funcționează la 1GHz trimite comenzi o dată la 1ns

- $X = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$
  $= \bar{A}(\bar{B}C + B\bar{C}) + A(\bar{B}\bar{C} + BC)$
  $= \bar{A}(B \oplus C) + A\overline{(B \oplus C)}$
  $= A \oplus B \oplus C$

- N*-? sunt mai rapide și mai mici ca suprafață decât *-?
- *-2 sunt mai rapide și mai mici ca suprafață decât *-4

motivul: unele tehnologii sunt bazate pe tranzistoare CMOS, iar circuitele analogice sunt mai eficiente pe logică negată (mai puține componente electrice)

- **este o unitate separată de calcul**
- **este conectată la un BUS de comunicare**
- **complementează capacitatea de calcul a unui CPU**
- **conțin un număr mare unități de procesare separate**
  - perfecte pentru procesarea (uniformă) unui bloc mare de date
- **sunt specializați pe un anumit set de instrucțiuni**
  - în general, nu pot realiza tot ce poate un CPU
  - dar ce pot executa, executa mult mai repede
    - SIMD/MIMD
    - hardware multithreading
    - Instruction Level Parallelism (ILP)
- **are propria sa memorie (on chip)**
  - este comparabilă cu cea a CPU (dar de obicei mai mică)
  - ierarhizare memoriei este mai simplă
  - timpii de acces nu sunt cea mai importantă caracteristică
  - bandwidth-ul (lățimea benzii de acces) este mai importantă

GPU

- **excelent pentru clasa de probleme "embarrassingly parallel"**
  - operații matrice-vector, matrice-matrice
    - calculul transformatei Fourier
  - procesarea imaginilor
  - convolutional neural networks (CNN) pentru Machine Learning
  - căutări exhaustive (brute force search)
    - căutarea hyperparametrilor
  - crypto mining
  - integrare numerică
  - simulări fizice cu scenarii diferite (condiții inițiale diferite)
  - raytracing
- **în general, este foarte dificil să definim clar ce înseamnă performanță în contextul arhitecturii calculatoarelor**

- **pentru CPU**
  - frecvența (*clock rate*), ex. 4GHz
  - ciclu de ceas (*clock cycle*), ex. la fiecare 250 pico secunde (ps)
    - $(\text{frecvența})^{-1}$

- **CPU time pentru A = ciclii de ceas pentru A / frecvența**
  - deci, putem micșora timpul de execuție pentru A dacă
    - reducem numărul de ciclii de ceas necesar pentru a executa A
    - mărim frecvența procesorului

- **ce face un CPU?**
  - execută instrucțiuni
  - câți ciclii de ceas sunt necesari pentru a executa o instrucțiune?
  - vă reamintesc, nu toate instrucțiunile sunt la fel de "dificile"

operația	instrucțiuni	# cicli
operații întregi/biți	add, sub, and, or, xor, sar, sal, lea, etc.	1
înmulțirea întregilor	mul, imul	3
împărțirea întregilor	div, idiv	depinde (20–80)
adunare floating point	addss, addsd	3
înmulțire floating point	mulss, mulsd	5
împărțire floating point	divss, divsd	depinde (20–80)
fused-multiply-add floating point	vfmass, vfmasd	5

- **ciclii de ceas pentru A = număr instrucțiuni în A × număr ciclii necesar pentru a executa o instrucțiune (în medie)**

- fie ***x***, ***y***, ***z*** **vectori care sunt de tipul *double***

operația	timpul
z[i] = x[i]	51.5 ps
z[i] += x[i]	60 ps
z[i] = x[i] + y[i]	69 ps
z[i] += x[i] + y[i]	74.5 ps
z[i] = x[i] × y[i]	75 ps
z[i] += x[i] × y[i]	93 ps
z[i] = x[i] / y[i]	1120 ps
z[i] = sqrt(x[i])	2050 ps
z[i] = log(x[i])	4200 ps
z[i] = exp(x[i])	5600 ps
z[i] = rand()	1850 ps

cum măsurăm timpul de execuție?

- pentru o secvență de cod
- pentru un executabil (Linux: time ./program)

```
#include <time.h>

clock_t start, end;
double cpu_time_used;

start = clock();
...
end = clock();
cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
```

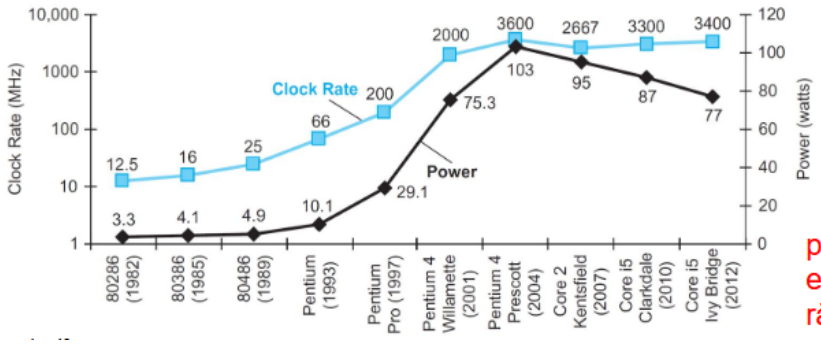
- **combinăm ultimele două relații**

CPU time pentru A = număr instrucțiuni în A × număr ciclii necesar pentru a executa o instrucțiune (în medie) / frecvența

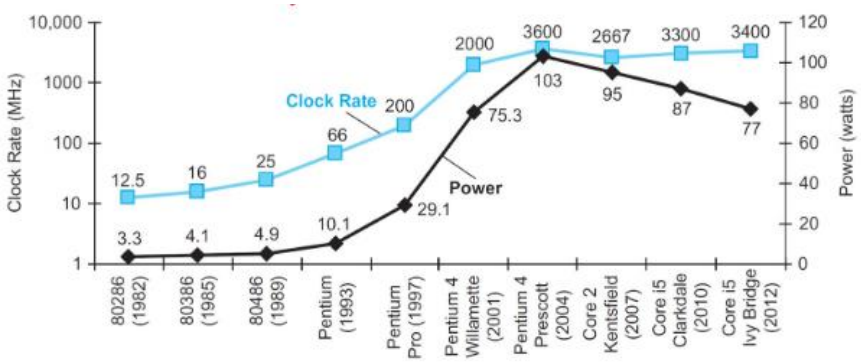
- **ce componentă ce afectează?**
  - algoritmul: număr instrucțiuni în A, în anumite situații speciale număr ciclii necesar pentru a executa o instrucțiune (în medie)
  - limbajul de programare: număr instrucțiuni în A, număr ciclii necesar pentru a executa o instrucțiune (în medie)
  - compilatorul: număr instrucțiuni în A, număr ciclii necesar pentru a executa o instrucțiune (în medie)
  - ISA: număr instrucțiuni în A, număr ciclii necesar pentru a executa o instrucțiune (în medie), frecvența

- **situația cu instrucțiunile e complicata**
  - poți executa puține instrucțiuni, dar unele pot avea nevoie de mai mulți ciclii de calcul pe CPU (deci este un trade-off)

- **situația cu frecvența procesorului e simplă**
  - frecvență mai mare → performanță mai ridicată
  - adică, mereu vrem să executăm mai repede
  - care e problema? de ce nu avem procesoare la 10GHz?



problema este energia consumată, răcirea sistemului



- **puterea = O(tensiune^2 × frecvență)**
- **uitați-vă pe grafic:**
  - am mers de la 25MHz la 2000MHz (×100)
  - în același timp puterea a mers de la 5W la 103W (×20)
  - cum e posibil?
    - tensiunea de funcționare a circuitelor a scăzut de la 5V la 1V
    - de ce nu putem merge mult sub 1V? zgomot!

- **mai sunt două probleme la 10GHz**
  - toate celelalte componente hardware ar trebui să funcționeze la viteze comparabile (altfel, CPU așteaptă mereu)
  - (1 / (10 GHz)) * (299 792 458 (m / s)) = 2.99 centimetri
    - ce semnificație are acest calcul?
    - este distanța (ideală, maximă) pe care o poate parcurge un semnal electric emis de un CPU care funcționează la 10GHz înainte ca același CPU să mai poată emite un nou semnal

mai mulți tranzistori = CPU
poate face mai multe lucruri
(simultan) și mai complexe

- **tehnologia curentă (dimensiunea tranzistorilor)**
  - 5nm
  - 5nm = 50 Å (Angstrom)
  - 1 atom are un diametru de aproximativ 1 Angstrom
  - intervine mecanica cuantică
    - nu mai știm exact unde este electronul
    - tranzistorul nu mai are cum să funcționeze corect (mereu)
    - nu avem cum să mai reducem dimensiunea tranzistorului și să păstrăm fiabilitatea sa
  - observație: calculatoarele cuantice nu au legatură cu această problemă

PERFORMANȚA PER WATT

- **un criteriu nou de performanță: cantitatea de energie consumată**
- **extrem de important pentru dispozitive pe baterie:**
  - laptop-uri
  - tablete
  - telefoane inteligente
  - sisteme embedded, sisteme Internet of Things (IoT)
- **toate aceste dispozitive își determină dinamic ciclul de ceas pentru a balansa consum de energie, răcire și performanță**
- **în general, aici RISC domină clar CISC**

- **am văzut deja că în multe situații compilatorul face multe optimizări pentru noi**
  - se asigură că rezultatul este același
  - dacă calculul este determinist, compilatorul calculează răspunsul final și în pune explicit în fișierul compilat
  - compilatorul alege ce funcții să apeleze (exemplu: printf vs. puts)
  - compilatorul alege dacă și cum să folosească instrucțiuni speciale (exemple: SIMD, div/mul, etc.)

- **în general, compilatorul își face treaba foarte bine**
  - compilatorul e mai capabil decât mulți programatori
  - compilatorele sunt îmbunătățite de decenii, deci conțin multe structuri/trucuri pentru a optimiza codul vostru
  - poate să transforme codul vostru în Assembly care logic este mai dificil de citit/debug pentru oameni, dar mai eficient ca instrucțiuni
  - compilatorul nu este perfect, nu este mereu corect
  - compilatorul nu poate (deocamdată) înlocui un programator capabil

- **de cele mai multe ori există o balanță între performanța și elemente de securitate în software**
- **de ce?**
  - verificări suplimentare de input de la user
  - verificări suplimentare la dimensiunea variabilelor/bufferelor
  - randomizarea adreselor de memorie este o metodă care sporește securitatea sistemului
  - criptare/decriptare date
- **practic, performanța unui sistem de calcul se află**
  - simulând încărcarea uzuală prezisă (*predicted workload*)
  - rularea unor programe standard de test pentru măsurarea performanței (*benchmarking*)
- **Standard Performance Evaluation Corporation (SPEC)**
  - benchmarking standardizat
  - evaluează
    - performanța de calcul
    - consumul de energie
- **este această cantitate una deterministă?**
  - dacă rulez același program (executabil) pe același sistem de calcul (arhitectura completă) cu exact aceleași setări (de exemplu folosește SIMD, etc.) și stare inițială (memorie RAM, cache inițializată la fel), am același timp de rulare?
  - NU
  - de ce?
  - nu uitați de detectarea și corectarea erorilor
    - deci, sistemul vostru de calcul nu face exact aceleași operații niciodată (cu probabilitate mare)